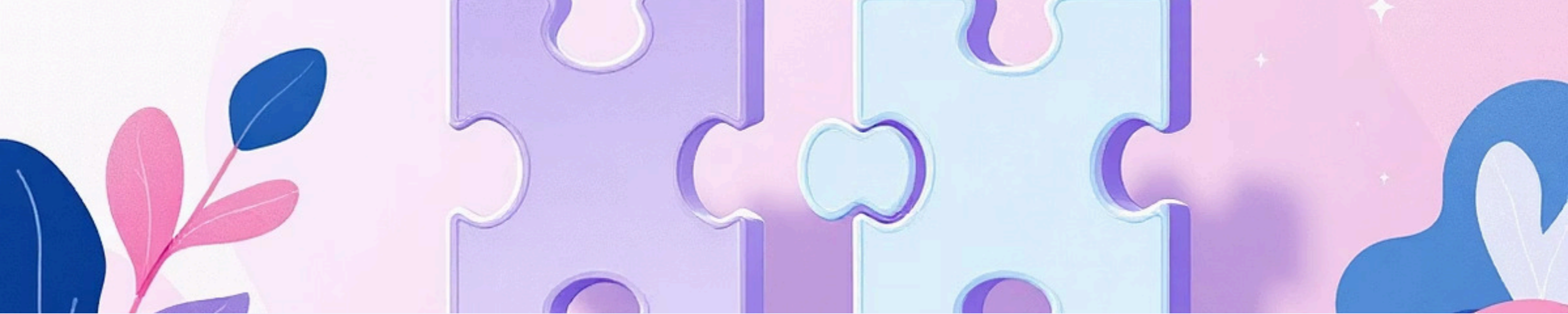


# Data Imputation in scikit-learn and pandas

Advanced Techniques for Handling Missing Data in ML Pipelines





# What is Imputation?

Imputation is the process of filling missing or null values with estimated ones, rather than dropping rows or columns entirely. This technique is fundamental to maintaining data integrity in machine learning workflows.

## Algorithm Compatibility

Most ML algorithms cannot process NaN values and require complete datasets for training and prediction.

## Data Preservation

Avoids unnecessary data loss and potential bias that comes from simply removing incomplete observations.

## Dataset Consistency

Keeps your dataset consistent, usable, and ready for modelling without sacrificing valuable information.

# SimpleImputer: Mean Strategy

The mean strategy replaces missing numerical values with the **arithmetic mean** of the column. This approach works best when your data follows a normal distribution without significant skewness.

## When to Use

- Data is normally distributed with no strong skew
- Missing values are few and occur randomly (MCAR)
- You need a quick, baseline imputation method

## Strengths & Limitations

- ✓ Easy to implement and computationally fast
- ✓ Maintains the central tendency of the distribution
- ✗ Highly affected by outliers, which can shift the mean significantly



## Code Example

```
from sklearn.impute import
SimpleImputer
import pandas as pd

df = pd.DataFrame({
    'Age': [25, 30, None, 35, 40]
})

imputer = SimpleImputer(
    strategy='mean'
)

df['Age'] =
imputer.fit_transform(
    df[['Age']]
)
```



# SimpleImputer: Median Strategy

The median strategy fills missing numeric values with the **middle value** when data is sorted. This is a robust alternative to the mean, particularly effective for skewed distributions or datasets containing outliers.

## Ideal Use Cases

- Data exhibits strong skewness (left or right)
- Dataset contains outliers that would distort the mean
- Working with income, price, or other right-skewed variables

## Advantages

- ✓ Robust to outliers and extreme values
- ✓ Works better for non-normal distributions
- ✓ Preserves data structure without artificial shifts

## Considerations

- ✗ May slightly distort relationships in symmetric data
- ✗ Less intuitive than mean for normally distributed features



```
imputer = SimpleImputer(  
    strategy='median'  
)  
  
df['Salary'] = imputer.fit_transform(  
    df[['Salary']]  
)
```



# SimpleImputer: Most Frequent Strategy



## What It Does

Replaces missing values with the **most common (mode)** value in the column. This is the go-to strategy for categorical features.



## Best Applications

Perfect for categorical features like Gender, City, Department, or Product Category where you want to fill with the most likely value.



## Key Benefits

Simple, intuitive approach that maintains the dominant category pattern in your data. Easy to explain to stakeholders.

## Implementation Example

```
imputer = SimpleImputer(  
    strategy='most_frequent'  
)  
  
df['Gender'] = imputer.fit_transform(  
    df[['Gender']]  
)
```

## Important Caveats

- ⚠ Can overrepresent already frequent categories, creating bias
- ⚠ Not suitable for continuous numeric variables

# SimpleImputer: Constant Value Strategy

This strategy fills missing data with a **fixed, predefined value** such as 'Unknown', 0, 'N/A', or any other constant you specify. It's particularly useful when missingness itself carries meaning.

## When to Use

- Categorical columns where missing = "not applicable" or "unknown"
- Features where you want to explicitly track missingness
- ID fields or reference columns that need placeholder values

## Code Example

```
imputer =  
SimpleImputer(  
  
    strategy='constant',  
  
    fill_value='Unknown'  
)  
  
df['City'] =  
imputer.fit_transform(  
    df[['City']]  
)
```

## Advantages

- ✓ Easy to interpret and explain
- ✓ Explicitly highlights missingness as a separate category

## Limitations

- ✗ Introduces an artificial category that didn't exist originally
- ✗ Can mislead models if not handled properly downstream



# KNN Imputer: Neighbourhood-Based Estimation

The KNN Imputer estimates missing values based on **K-nearest neighbours**—it identifies the most similar rows (using distance metrics) and uses their values to intelligently fill gaps. This approach leverages relationships between features for more accurate imputation.

## How It Works

1. Identifies K most similar rows based on available features
2. Computes distance metrics (typically Euclidean)
3. Averages values from nearest neighbours to fill missing data

## Ideal Scenarios

- Numerical data where features are correlated
- Missing values depend on patterns in other variables
- Medium-sized datasets with structured relationships



```
from sklearn.impute import KNNImputer

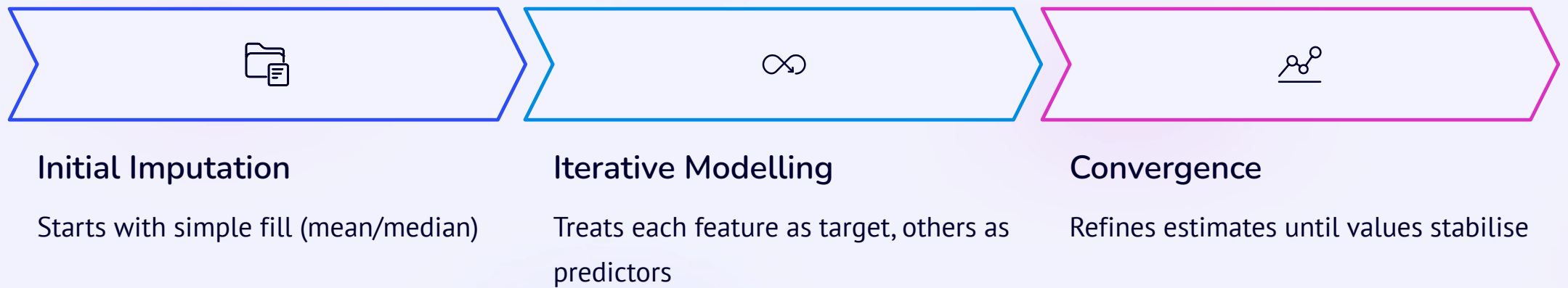
imputer = KNNImputer(
    n_neighbors=3
)

df_imputed = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
```

## Performance Trade-offs

- ✓ Captures complex feature relationships effectively
- ✓ More accurate than simple statistical methods
- ✗ Computationally slow on large datasets
- ✗ Cannot handle categorical data directly

# Iterative Imputer: Model-Based Prediction



The Iterative Imputer uses a **regression model** to predict missing values iteratively. Each feature with missing values is modelled as a function of other features, creating sophisticated, data-driven estimates.

## When to Choose This Method

- Features are highly correlated with each other
- Simple mean/median approaches are insufficient
- You need the most accurate imputation possible
- Dataset has complex, non-linear relationships

## Strengths

- ✓ Handles complex feature dependencies
- ✓ Most accurate for numeric data
- ✓ Works with both linear and non-linear patterns

## Drawbacks

- ✗ Computationally expensive and time-intensive
- ✗ Risk of overfitting on small datasets
- ✗ Requires careful hyperparameter tuning



```
from sklearn.experimental import
enable_iterative_imputer
from sklearn.impute import IterativeImputer

imputer = IterativeImputer()
df_imputed = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
```



# Comparison of sklearn Imputation Methods

| Technique | Best For             | Strength             | Limitation            |
|-----------|----------------------|----------------------|-----------------------|
| Mean      | Normal numeric data  | Simple, fast         | Sensitive to outliers |
| Median    | Skewed data          | Robust to outliers   | May lose precision    |
| Mode      | Categorical features | Intuitive            | Can bias common class |
| Constant  | IDs / Categories     | Easy, explicit       | Adds artificial class |
| KNN       | Correlated data      | Learns from patterns | Slow on large data    |
| Iterative | Complex data         | Most accurate        | Heavy computation     |

Choose your imputation strategy based on data distribution, feature relationships, computational resources, and accuracy requirements. Often, experimenting with multiple methods and comparing downstream model performance is the best approach.

# Imputation with pandas: Quick Reference

## Common pandas Methods

```
df['col'].fillna(df['col'].mean())
```

# Mean imputation

```
df['col'].fillna(df['col'].median())
```

# Median imputation

```
df['col'].fillna(df['col'].mode()[0])
```

# Mode imputation

```
df.fillna(method='ffill')
```

# Forward fill

```
df.fillna(method='bfill')
```

# Backward fill



## Key Takeaways

✓ Fast and convenient for small data cleaning tasks

✓ Easy syntax for quick exploratory fixes

⚠ Not integrated with ML pipelines

**Recommendation:** Prefer sklearn imputers for production ML workflows to ensure reproducibility and pipeline compatibility.